

AU Property Data Collection & Consolidation

Step-by-Step, Screen-to-Screen Build Guide
SSIS Extract-Transform-Load into a SQL Server Data Warehouse

Task Reference: **Task 5 - AU Property Data Collection and Consolidation**

BI Developer / Data Analyst Advanced Sprint

Prepared for: Srikanth | Date: 13 July 2026

This guide walks through every screen you will use - from creating the Data Warehouse in SQL Server Management Studio, to building the SSIS package in Visual Studio, to the final T-SQL consolidation and validation - based directly on the 14 raw Excel files supplied in the RawDataSet folder.

Contents

- 1** How This Guide Works
- 2** Solution Architecture
- 3** Source Data Inventory
- 4** Data Quality Findings (from profiling the raw files)
- 5** Step 1 - Create the Data Warehouse & Schemas
- 6** Step 2 - Create the Staging Tables
- 7** Step 3 - Create the SSIS Project in Visual Studio
- 8** Step 4 - Configure Connection Managers
- 9** Step 5 - Lay Out the Control Flow
- 10** Step 6 - Build a Data Flow Task (worked example)
- 11** Step 7 - Repeat the Pattern for the Remaining Files
- 12** Step 8 - Precedence Constraints & Execute the Package
- 13** Step 9 - Consolidate Staging -> Data Warehouse (T-SQL)
- 14** Step 10 - Validate the Load
- 15** Step 11 - Save, Document & Submit
- 16** Appendix A - Full Staging Table DDL
- 17** Appendix B - Full Consolidation & Validation SQL
- 18** Appendix C - Screen Navigation Quick-Reference

1. How This Guide Works

Each build step below is broken into **numbered screens** - one screen = one dialog, tab, or pane you will actually see in SQL Server Management Studio (SSMS) or Visual Studio. Follow them in order and you will move through the tool exactly the way the finished package was designed to be built.

- **Grey boxes** contain exact T-SQL / DDL you can copy-paste and run.
- **Orange DATA QUALITY NOTE boxes** flag a real issue found in this specific dataset (inconsistent casing, missing columns, duplicate rows, etc.) and the decision made to handle it.
- **Teal SCREEN boxes** are the click-by-click navigation steps.

PREREQUISITE: This guide assumes SSMS, Visual Studio with the SQL Server Integration Services (SSDT) extension, and access to a SQL Server instance are already installed and that you can reach the RawDataSet folder from your machine.

2. Solution Architecture

Per the task instructions, data flows through two layers inside the Data Warehouse: a **staging layer** that loads each source file with minimal changes (SSIS), and a **consolidated layer** that unions the three states together per domain, cleans the data, and standardizes it for reporting (T-SQL). This matches the brief's own suggestion: consider using SSIS tasks for data consolidation, or consider using SQL for data consolidation - this guide uses SSIS for extraction/loading and SQL for consolidation, since it is the most maintainable split for 14 source files with inconsistent schemas.

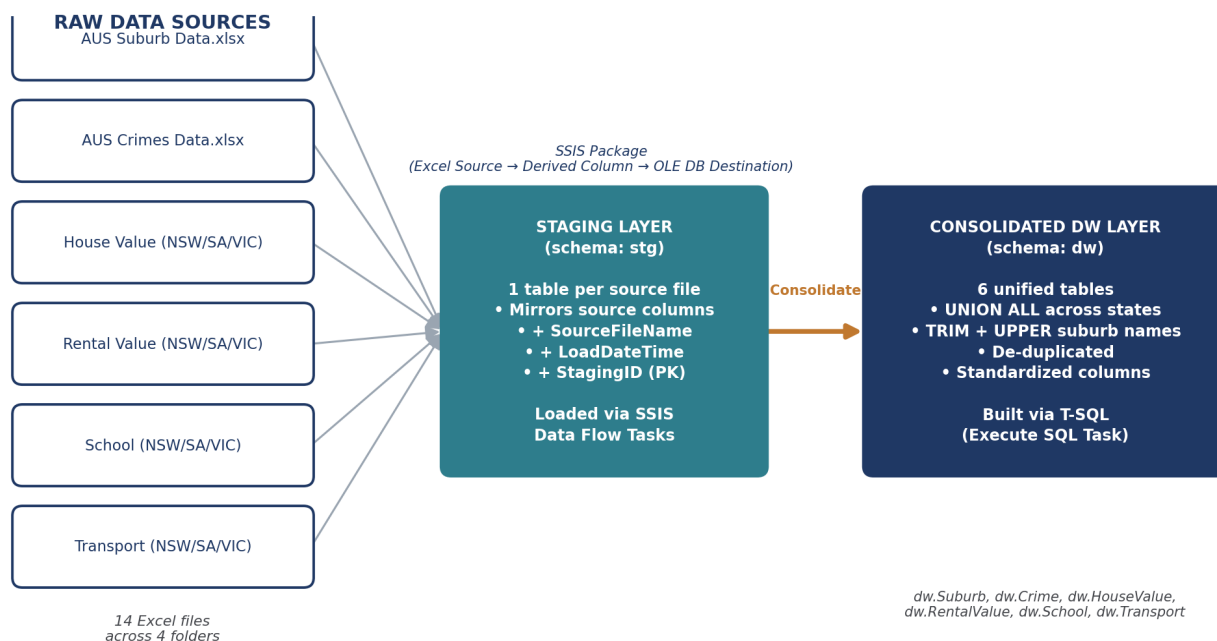


Figure 1. Layered ETL architecture: raw Excel -> staging (stg) -> consolidated Data Warehouse (dw).

3. Source Data Inventory

All 14 files were opened and profiled directly. Row counts exclude header rows.

Folder	File	Data Rows	Columns
(root)	AUS Suburb Data.xlsx	15,421	suburb, city, state, state_code, lat, lon
Crime	AUS Crimes Data.xlsx	360,999	State, suburb, Postcode, Offence category, Offence subcategory, Recorded Incidents
House Value	NSW House Value Data.xlsx	4,672	state_code, Suburb, Postcode, HouseValue
House Value	SA House Value Data.xlsx	416	state_code, Suburb, HouseValue
House Value	VIC House Value Data.xlsx	976	state_code, SUBURB, HouseValue
Rental Value	NSW Rental Value Data.xlsx	11,341	state_code, Suburb, RentalHouseType, RentalAmount
Rental Value	SA Rental Value Data.xlsx	2,172	state_code, Suburb, RentalHouseType, RentalAmount
Rental Value	VIC Rental Value Data.xlsx	1,206	state_code, Suburb, RentalHouseType, RentalAmount
School	NSW School Data.xlsx	2,182	state_code, town_suburb, postcode, school_code, school_name, SchoolType, address
School	SA School Data.xlsx	2,232	state_code, suburb, Post Code, SchoolCode, school_name, SchoolType, Address
School	VIC School Data.xlsx	2,159	state_code, suburb, Postal_Postcode, SchoolCode, school_name, SchoolType, Address
Transport	NSW Transport Data.xlsx	613	state_code, suburb, stop_name, mode, stop_lat, stop_long
Transport	SA Transport Data.xlsx	5,079	state_code, suburb, stop_name, mode, stop_lat, stop_lon
Transport	VIC Transport Data.xlsx	6,368	state_code, suburb, stop_name, mode, stop_lat, stop_lon

4. Data Quality Findings (from profiling the raw files)

These are concrete issues found by opening the actual files - not hypothetical ones - and the guide handles each of them explicitly in Steps 6 and 9.

4.1 Inconsistent schemas across states, within the same domain

- **House Value:** NSW has a *Postcode* column; SA and VIC do not. The three files cannot share one staging table structure - each gets its own staging table, unioned later with Postcode set to NULL for SA/VIC.
- **School:** the postcode column is named *postcode* (NSW), *Post Code* (SA) and *Postal_Postcode* (VIC); the suburb column is *town_suburb* in NSW vs *suburb* elsewhere. Column order is identical across all three, but names must be aliased when consolidating.
- **Transport:** the longitude column is named *stop_long* in NSW but *stop_lon* in SA/VIC.

4.2 Inconsistent suburb-name casing and spacing

- House Value suburb names are Title Case in NSW but ALL CAPS in SA and VIC (e.g. Abbotsbury vs ABERFOYLE PARK).
- NSW School data has untrimmed suburb names in **1,571 of 2,182 rows (72%)** - e.g. 'Toongabbie ' with a trailing space - which would break any join or group-by until trimmed.
- Standardizing suburb names with `UPPER(LTRIM(RTRIM(...)))` and testing against the master AUS Suburb Data list confirmed a **100% match rate** (416/416) for SA House Value once normalized this way - confirming this is the right standardization rule to use throughout.

4.3 Duplicate rows

- VIC House Value Data.xlsx contains **976 rows but only 724 distinct rows** - 252 exact duplicates. These are removed with `SELECT DISTINCT` during consolidation (Step 9), not during staging (staging should always mirror the raw source exactly, for auditability).

5. Step 1 - Create the Data Warehouse & Schemas

Open **SQL Server Management Studio (SSMS)** and connect to your SQL Server instance.

SCREEN 1 SSMS - Object Explorer

Right-click **Databases** -> **New Database...** -> set Database name to **AUPROPERTYDW** -> **OK**.

SCREEN 2 SSMS - New Query window

Right-click **AUPROPERTYDW** -> **New Query**, then run the script below to create the two schemas that separate raw loads from cleaned data.

```
USE AUPROPERTYDW;
GO

CREATE SCHEMA stg; -- staging: one table per source file, minimal transformation
GO
CREATE SCHEMA dw;  -- consolidated: cleaned, unioned, standardized tables
GO
```

WHY TWO SCHEMAS: Keeping stg and dw separate lets you re-run the SSIS load at any time without touching the cleaned tables, and lets you trace any bad value in dw back to the exact source row in stg.

6. Step 2 - Create the Staging Tables

Still in the SSMS query window, create one staging table per source file (14 total). Each mirrors its source file's exact columns and adds three audit columns: StagingID (identity key), SourceFileName, and LoadDateTime. The full script for all 14 tables is in **Appendix A**; two representative examples are shown here.

```
CREATE TABLE stg.Suburb (
    StagingID      INT IDENTITY(1,1) PRIMARY KEY,
    suburb         NVARCHAR(100),
    city           NVARCHAR(100),
    state          NVARCHAR(50),
    state_code     NVARCHAR(5),
    lat            FLOAT,
    lon            FLOAT,
    SourceFileName NVARCHAR(255),
    LoadDateTime  DATETIME2 DEFAULT SYSDATETIME()
);

CREATE TABLE stg.HouseValue_NSW (
    StagingID      INT IDENTITY(1,1) PRIMARY KEY,
    state_code     NVARCHAR(5),
    Suburb         NVARCHAR(100),
    Postcode       INT,
    HouseValue     INT,
    SourceFileName NVARCHAR(255),
    LoadDateTime  DATETIME2 DEFAULT SYSDATETIME()
);
-- HouseValue_SA / HouseValue_VIC are identical but WITHOUT the Postcode column
-- (see Appendix A) -- this mirrors the source files exactly.
```

Run the full script from **Appendix A** now to create all 14 staging tables before continuing.

7. Step 3 - Create the SSIS Project in Visual Studio

SCREEN 1 Visual Studio - Start Window

File -> New -> Project... Search the templates box for Integration Services Project (requires the SSDT / Integration Services extension). Select it and click **Next**.

SCREEN 2 Visual Studio - Configure your new project

Project name: AUPROPERTY_ETL. Choose a Location on disk. Click **Create**. Visual Studio opens with an empty package named Package.dtsx.

SCREEN 3 Solution Explorer

Right-click **Package.dtsx** -> **Rename** -> AUPROPERTY_StagingLoad.dtsx. Click **Yes** when asked to update the package object's name too.

8. Step 4 - Configure Connection Managers

Connection Managers live in the pane at the bottom of the package designer.

SCREEN 1 Connection Managers pane - the Data Warehouse

1. Right-click the empty Connection Managers area -> **New OLE DB Connection...**
2. Click **New...**, enter your SQL Server name, choose Windows Authentication (or SQL auth), select database **AUPROPERTYDW**, click **Test Connection**, then **OK** twice.
3. Rename the connection manager to **OleDb_AUPROPERTYDW**.

SCREEN 2 Connection Managers pane - one Excel connection per source file

1. Right-click -> **New Connection...** -> choose **EXCEL** -> **Add...**
2. Browse to the file (e.g. RawDataSet\House Value\NSW House Value Data.xlsx), set Excel version to **Microsoft Excel 2007**, check **First row has column names** -> **OK**.
3. Rename the connection manager to something predictable, e.g. **XLS_HouseValue_NSW**.

TIP: Repeat Screen 2 for all 14 source files up front (14 Excel connection managers total) - it is faster to set them all up in one pass than to interleave it with building each Data Flow Task. Use the naming pattern **XLS_<Domain>_<State>** (e.g. **XLS_School_SA**, **XLS_Transport_VIC**) so the Appendix C reference table lines up.

9. Step 5 - Lay Out the Control Flow

Switch to the **Control Flow** tab. This is the overall shape of the package: a truncate step, five parallel sequence containers (one per domain), and a final consolidation step.

SCREEN 1 SSIS Toolbox -> Control Flow canvas

1. Drag an **Execute SQL Task** onto the canvas. Rename it 0 - Truncate Staging Tables. Double-click it, set Connection to OLEDB_AUPROPERTYDW, and paste 14 TRUNCATE TABLE stg.... statements (one per staging table) into the SQLStatement box.
2. Drag five **Sequence Containers** onto the canvas, one below the truncate task. Rename them: SEQ - Suburb & Crime, SEQ - House Value, SEQ - Rental Value, SEQ - School, SEQ - Transport.
3. Drag a green precedence-constraint arrow from the truncate task to each of the five containers.
4. Drag one more **Execute SQL Task** below all five containers. Rename it 9 - Consolidate Staging -> DW.
5. Drag a precedence arrow from each of the five containers into this final task. Double-click the last arrow that lands on the task, and in the Precedence Constraint Editor confirm **Multiple constraints -> Logical AND**. All constraints must evaluate to True - this makes the task wait for all five containers, not just the first to finish.

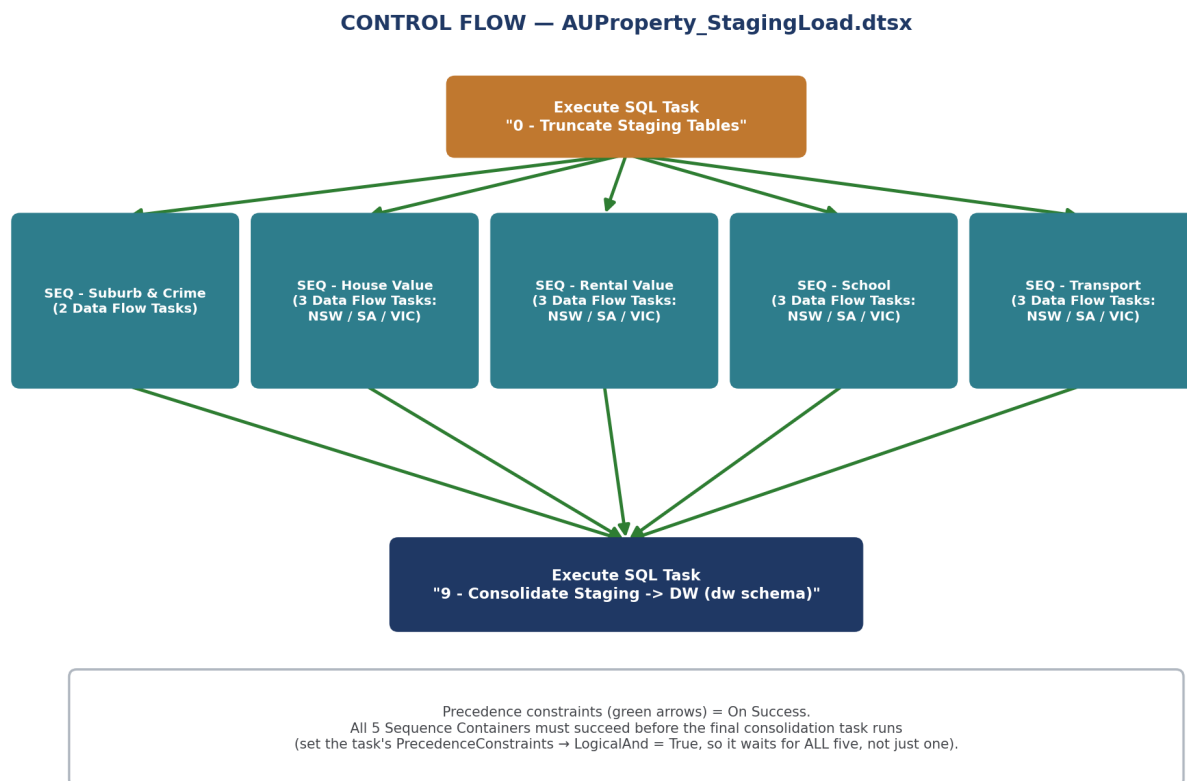


Figure 2. Target Control Flow layout for AUPROPERTY_StagingLoad.dtsx.

10. Step 6 - Build a Data Flow Task (worked example)

Build the **House Value - NSW** load first; the same 3-component pattern is then repeated for the other 13 files (Step 7).

SCREEN 1 Inside SEQ - House Value

Drag a **Data Flow Task** into the container. Rename it DFT - HouseValue NSW. Double-click it to open the **Data Flow** tab.

SCREEN 2 Data Flow tab - Source

1. Drag an **Excel Source** from the toolbox onto the canvas.
2. Double-click it. Connection Manager: XLS_HouseValue_NSW. Data access mode: **Table or view**. Name of the Excel sheet: Sheet1\$.
3. Go to the **Columns** page, confirm all four columns are ticked (state_code, Suburb, Postcode, HouseValue) -> **OK**.

SCREEN 3 Data Flow tab - Derived Column (cleansing at the point of load)

1. Drag a **Derived Column** transformation onto the canvas, below the source. Connect the blue arrow from Excel Source into it.
2. Double-click it and add three derived columns:

Derived Column Name	Derived Column	Expression
SourceFileName_Derived	<add as new column>	"NSW House Value Data.xlsx"
LoadDateTime_Derived	<add as new column>	GETDATE()
Suburb_Trimmed	Replace 'Suburb'	TRIM(Suburb)

SCREEN 4 Data Flow tab - Destination

1. Drag an **OLE DB Destination** onto the canvas. Connect the Derived Column output into it.
2. Double-click it. OLE DB connection manager: OLEDB_AUPropertyDW. Table: [stg].[HouseValue_NSW].
3. Open the **Mappings** page. Map state_code->state_code, Suburb_Trimmed->Suburb, Postcode->Postcode, HouseValue->HouseValue, SourceFileName_Derived->SourceFileName, LoadDateTime_Derived->LoadDateTime -> **OK**.

DATA QUALITY NOTE: Trimming happens here, at load time, so every staging table already has clean suburb text - casing standardization (UPPER) is deferred to the consolidation step (Step 9) since it is a cross-file rule, not a per-file one.

11. Step 7 - Repeat the Pattern for the Remaining Files

Right-click the three components you just built -> **Copy**, paste them into each remaining Data Flow Task, then just re-point the Excel Source connection manager and the OLE DB Destination table. The mapping below tells you exactly what to change for each of the other 13 files.

Sequence Container	Data Flow Task	Excel Connection Mgr	Destination Table
SEQ - Suburb & Crime	DFT - Suburb	XLS_Suburb	stg.Suburb
SEQ - Suburb & Crime	DFT - Crime	XLS_Crime	stg.Crime
SEQ - House Value	DFT - HouseValue NSW	XLS_HouseValue_NSW	stg.HouseValue_NSW (built above)
SEQ - House Value	DFT - HouseValue SA	XLS_HouseValue_SA	stg.HouseValue_SA
SEQ - House Value	DFT - HouseValue VIC	XLS_HouseValue_VIC	stg.HouseValue_VIC
SEQ - Rental Value	DFT - RentalValue NSW	XLS_RentalValue_NSW	stg.RentalValue_NSW
SEQ - Rental Value	DFT - RentalValue SA	XLS_RentalValue_SA	stg.RentalValue_SA
SEQ - Rental Value	DFT - RentalValue VIC	XLS_RentalValue_VIC	stg.RentalValue_VIC
SEQ - School	DFT - School NSW	XLS_School_NSW	stg.School_NSW
SEQ - School	DFT - School SA	XLS_School_SA	stg.School_SA
SEQ - School	DFT - School VIC	XLS_School_VIC	stg.School_VIC
SEQ - Transport	DFT - Transport NSW	XLS_Transport_NSW	stg.Transport_NSW
SEQ - Transport	DFT - Transport SA	XLS_Transport_SA	stg.Transport_SA
SEQ - Transport	DFT - Transport VIC	XLS_Transport_VIC	stg.Transport_VIC

DATA QUALITY NOTE: For **School** files, the source column names differ per state (e.g. SA's Post Code vs VIC's Postal_Postcode). In the Excel Source -> Columns page the external column names will show the file's own headers - that's fine; just map each one to the same staging table column position when you reach the Destination Mappings page. For **Transport**, map NSW's stop_long and SA/VIC's stop_lon both to the same destination column.

12. Step 8 - Precedence Constraints & Execute the Package

SCREEN 1 Control Flow tab

Confirm every Data Flow Task inside a Sequence Container is connected in the order you built them (or left in parallel - order does not matter within a container). Confirm the precedence arrows from Step 5 are all green (On Success).

SCREEN 2 Debug menu

Debug -> Start Debugging (or press **F5**). Watch each component turn yellow (running) then green (success) in sequence. A red component means a failure - right-click it -> **View Data Lineage** or check the **Progress/Execution Results** tab for the error text.

When every task is green, press **Shift+F5** to stop debugging.

SCREEN 3 SSMS - spot-check the load

Run `SELECT COUNT(*) FROM stg.HouseValue_NSW;` and compare it against the 4,672 data rows recorded in the Source Data Inventory (Section 3). Repeat for a couple of other staging tables before moving on.

13. Step 9 - Consolidate Staging -> Data Warehouse (T-SQL)

With all 14 staging tables loaded, create the six dw tables and populate them with cleaned, unioned data. You can run this as a second **New Query** in SSMS, or paste it into the 9 - Consolidate Staging -> DW Execute SQL Task from Step 5 so the whole pipeline runs in one click. The full script (DDL + inserts) is in **Appendix A** and **Appendix B**; the House Value union - the trickiest one, because of the missing Postcode column and the VIC duplicates - is shown here.

```
-- House Value: NSW has Postcode, SA/VIC do not -> NULL-fill.  
-- VIC source file has 252 duplicate rows -> DISTINCT removes them.  
INSERT INTO dw.HouseValue (StateCode, Suburb, Postcode, HouseValue)  
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(Suburb))), Postcode, HouseValue  
FROM stg.HouseValue_NSW  
UNION ALL  
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(Suburb))), NULL, HouseValue  
FROM stg.HouseValue_SA  
UNION ALL  
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(SUBURB))), NULL, HouseValue  
FROM stg.HouseValue_VIC;
```

DATA QUALITY NOTE: UPPER(LTRIM(RTRIM(...))) is the standardization rule used throughout this consolidation step. It was validated against the master dw.Suburb list in Section 4.2 with a 100% match rate on SA House Value, so it is safe to apply the same rule to every other domain.

14. Step 10 - Validate the Load

Run these checks in SSMS after consolidation to confirm the Data Warehouse is correct before calling the task complete. Full text also in Appendix B.

A. Row-count reconciliation (staging vs. consolidated)

```
SELECT 'HouseValue' AS Domain,
      (SELECT COUNT(*) FROM stg.HouseValue_NSW)
    + (SELECT COUNT(*) FROM stg.HouseValue_SA)
    + (SELECT COUNT(*) FROM stg.HouseValue_VIC)           AS StagingRows,
      (SELECT COUNT(*) FROM dw.HouseValue)                AS DWRows;
-- Expect DWRows to be ~252 lower than StagingRows (the VIC duplicates removed).
```

B. Null / blank key checks

```
SELECT COUNT(*) AS NullSuburbs FROM dw.HouseValue WHERE Suburb IS NULL OR Suburb = '';
```

C. Orphan suburb check (values that don't exist in the master suburb list)

```
SELECT DISTINCT hv.Suburb, hv.StateCode
FROM dw.HouseValue hv
LEFT JOIN dw.Suburb s ON hv.Suburb = s.Suburb AND hv.StateCode = s.StateCode
WHERE s.SuburbID IS NULL;
-- Investigate any rows returned -- likely a spelling variant needing an extra
-- cleansing rule before this data is trusted downstream.
```


15. Step 11 - Save, Document & Submit

1. In Visual Studio, **File -> Save All**, then close the solution.
2. Take a screenshot of the completed, all-green Control Flow (Step 8, Screen 2) as evidence the package runs end-to-end - keep it with your submission.
3. In SSMS, run the Step 10 validation queries one more time and save the results/screenshots.
4. Per the task brief: upload your final SSIS project folder (the .sln/.dtproj files) and your validation evidence to your Google Drive for review.
5. Update your task-tracking Google Sheet to mark Task 5 - AU Property Data Collection and Consolidation as complete.

Appendix A - Full Staging Table DDL

```

USE AUPROPERTYDW;
GO
CREATE SCHEMA stg;
GO
CREATE SCHEMA dw;
GO

-- ===== NATIONAL FILES =====
CREATE TABLE stg.Suburb (
    StagingID          INT IDENTITY(1,1) PRIMARY KEY,
    suburb             NVARCHAR(100),
    city              NVARCHAR(100),
    state             NVARCHAR(50),
    state_code        NVARCHAR(5),
    lat               FLOAT,
    lon               FLOAT,
    SourceFileName     NVARCHAR(255),
    LoadDateTime     DATETIME2 DEFAULT SYSDATETIME()
);

CREATE TABLE stg.Crime (
    StagingID          INT IDENTITY(1,1) PRIMARY KEY,
    State             NVARCHAR(5),
    suburb            NVARCHAR(100),
    Postcode          INT,
    [Offence category] NVARCHAR(100),
    [Offence subcategory] NVARCHAR(150),
    [Recorded Incidents] INT,
    SourceFileName     NVARCHAR(255),
    LoadDateTime     DATETIME2 DEFAULT SYSDATETIME()
);

-- ===== HOUSE VALUE =====
CREATE TABLE stg.HouseValue_NSW (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    Suburb NVARCHAR(100), Postcode INT, HouseValue INT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.HouseValue_SA (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    Suburb NVARCHAR(100), HouseValue INT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.HouseValue_VIC (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    SUBURB NVARCHAR(100), HouseValue INT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);

-- ===== RENTAL VALUE (same shape x3) =====
CREATE TABLE stg.RentalValue_NSW (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), Suburb NVARCHAR(100),
    RentalHouseType NVARCHAR(50), RentalAmount DECIMAL(10,2),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.RentalValue_SA (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), Suburb NVARCHAR(100),
    RentalHouseType NVARCHAR(50), RentalAmount DECIMAL(10,2),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.RentalValue_VIC (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), Suburb NVARCHAR(100),
    RentalHouseType NVARCHAR(50), RentalAmount DECIMAL(10,2),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);

-- ===== SCHOOL (column names differ per state) =====
CREATE TABLE stg.School_NSW (

```

```

    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    town_suburb NVARCHAR(100), postcode INT, school_code INT, school_name NVARCHAR(200),
    SchoolType NVARCHAR(50), address NVARCHAR(200),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.School_SA (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    suburb NVARCHAR(100), [Post Code] INT, SchoolCode INT, school_name NVARCHAR(200),
    SchoolType NVARCHAR(50), Address NVARCHAR(200),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.School_VIC (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5),
    suburb NVARCHAR(100), Postal_Postcode INT, SchoolCode INT, school_name NVARCHAR(200),
    SchoolType NVARCHAR(50), Address NVARCHAR(200),
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);

-- ===== TRANSPORT (stop_long vs stop_lon) =====
CREATE TABLE stg.Transport_NSW (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), suburb NVARCHAR(100),
    stop_name NVARCHAR(200), mode NVARCHAR(50), stop_lat FLOAT, stop_long FLOAT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.Transport_SA (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), suburb NVARCHAR(100),
    stop_name NVARCHAR(200), mode NVARCHAR(50), stop_lat FLOAT, stop_lon FLOAT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);
CREATE TABLE stg.Transport_VIC (
    StagingID INT IDENTITY(1,1) PRIMARY KEY, state_code NVARCHAR(5), suburb NVARCHAR(100),
    stop_name NVARCHAR(200), mode NVARCHAR(50), stop_lat FLOAT, stop_lon FLOAT,
    SourceFileName NVARCHAR(255), LoadDateTime DATETIME2 DEFAULT SYSDATETIME()
);

-- ===== CONSOLIDATED DW LAYER =====
CREATE TABLE dw.Suburb (
    SuburbID INT IDENTITY(1,1) PRIMARY KEY, Suburb NVARCHAR(100), City NVARCHAR(100),
    State NVARCHAR(50), StateCode NVARCHAR(5), Latitude FLOAT, Longitude FLOAT
);
CREATE TABLE dw.Crime (
    CrimeID INT IDENTITY(1,1) PRIMARY KEY, StateCode NVARCHAR(5), Suburb NVARCHAR(100),
    Postcode INT, OffenceCategory NVARCHAR(100), OffenceSubcategory NVARCHAR(150),
    RecordedIncidents INT
);
CREATE TABLE dw.HouseValue (
    HouseValueID INT IDENTITY(1,1) PRIMARY KEY, StateCode NVARCHAR(5), Suburb NVARCHAR(100),
    Postcode INT NULL, HouseValue INT
);
CREATE TABLE dw.RentalValue (
    RentalValueID INT IDENTITY(1,1) PRIMARY KEY, StateCode NVARCHAR(5), Suburb NVARCHAR(100),
    RentalHouseType NVARCHAR(50), RentalAmount DECIMAL(10,2)
);
CREATE TABLE dw.School (
    SchoolID INT IDENTITY(1,1) PRIMARY KEY, StateCode NVARCHAR(5), Suburb NVARCHAR(100),
    Postcode INT, SchoolCode INT, SchoolName NVARCHAR(200), SchoolType NVARCHAR(50),
    Address NVARCHAR(200)
);
CREATE TABLE dw.Transport (
    TransportID INT IDENTITY(1,1) PRIMARY KEY, StateCode NVARCHAR(5), Suburb NVARCHAR(100),
    StopName NVARCHAR(200), Mode NVARCHAR(50), StopLat FLOAT, StopLong FLOAT
);

```

Appendix B - Full Consolidation & Validation SQL

```

TRUNCATE TABLE dw.Suburb;
TRUNCATE TABLE dw.Crime;
TRUNCATE TABLE dw.HouseValue;
TRUNCATE TABLE dw.RentalValue;
TRUNCATE TABLE dw.School;
TRUNCATE TABLE dw.Transport;

-- 1) Suburb (master geo reference)
INSERT INTO dw.Suburb (Suburb, City, State, StateCode, Latitude, Longitude)
SELECT DISTINCT UPPER(LTRIM(RTRIM(suburb))), LTRIM(RTRIM(city)), LTRIM(RTRIM(state)),
    state_code, lat, lon
FROM stg.Suburb;

-- 2) Crime
INSERT INTO dw.Crime (StateCode, Suburb, Postcode, OffenceCategory, OffenceSubcategory, RecordedIncidents)
SELECT State, UPPER(LTRIM(RTRIM(suburb))), Postcode,
    [Offence category], [Offence subcategory], [Recorded Incidents]
FROM stg.Crime;

-- 3) House Value (NULL-fill missing Postcode; DISTINCT removes VIC's 252 duplicate rows)
INSERT INTO dw.HouseValue (StateCode, Suburb, Postcode, HouseValue)
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(Suburb))), Postcode, HouseValue FROM stg.HouseValue_NSW
UNION ALL
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(Suburb))), NULL, HouseValue FROM stg.HouseValue_SA
UNION ALL
SELECT DISTINCT state_code, UPPER(LTRIM(RTRIM(SUBURB))), NULL, HouseValue FROM stg.HouseValue_VIC;

-- 4) Rental Value (identical structure across states)
INSERT INTO dw.RentalValue (StateCode, Suburb, RentalHouseType, RentalAmount)
SELECT state_code, UPPER(LTRIM(RTRIM(Suburb))), RentalHouseType, RentalAmount FROM stg.RentalValue_NSW
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(Suburb))), RentalHouseType, RentalAmount FROM stg.RentalValue_SA
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(Suburb))), RentalHouseType, RentalAmount FROM stg.RentalValue_VIC;

-- 5) School (column names aliased to a common target)
INSERT INTO dw.School (StateCode, Suburb, Postcode, SchoolCode, SchoolName, SchoolType, Address)
SELECT state_code, UPPER(LTRIM(RTRIM(town_suburb))), postcode, school_code, school_name, SchoolType, address
FROM stg.School_NSW
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(suburb))), [Post Code], SchoolCode, school_name, SchoolType, Address
FROM stg.School_SA
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(suburb))), Postal_Postcode, SchoolCode, school_name, SchoolType, Address
FROM stg.School_VIC;

-- 6) Transport (stop_long in NSW vs stop_lon in SA/VIC -> same destination column)
INSERT INTO dw.Transport (StateCode, Suburb, StopName, Mode, StopLat, StopLong)
SELECT state_code, UPPER(LTRIM(RTRIM(suburb))), stop_name, mode, stop_lat, stop_long FROM stg.Transport_NSW
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(suburb))), stop_name, mode, stop_lat, stop_lon FROM stg.Transport_SA
UNION ALL
SELECT state_code, UPPER(LTRIM(RTRIM(suburb))), stop_name, mode, stop_lat, stop_lon FROM stg.Transport_VIC;

-- ===== VALIDATION =====
-- Row-count reconciliation
SELECT 'HouseValue' AS Domain,
    (SELECT COUNT(*) FROM stg.HouseValue_NSW) + (SELECT COUNT(*) FROM stg.HouseValue_SA)
    + (SELECT COUNT(*) FROM stg.HouseValue_VIC) AS StagingRows,
    (SELECT COUNT(*) FROM dw.HouseValue) AS DWRows;

-- Null / blank key checks
SELECT COUNT(*) AS NullSuburbs FROM dw.HouseValue WHERE Suburb IS NULL OR Suburb = '';

-- Orphan suburb check against the master list
SELECT DISTINCT hv.Suburb, hv.StateCode
FROM dw.HouseValue hv
LEFT JOIN dw.Suburb s ON hv.Suburb = s.Suburb AND hv.StateCode = s.StateCode

```

```
WHERE s.SuburbID IS NULL;
```

Appendix C - Screen Navigation Quick-Reference

Action	Menu Path
Create database	SSMS -> Object Explorer -> right-click Databases -> New Database...
Open query editor	SSMS -> right-click database -> New Query
New SSIS project	Visual Studio -> File -> New -> Project -> Integration Services Project
Rename a package	Solution Explorer -> right-click .dtsx -> Rename
New OLE DB connection	Connection Managers pane -> right-click -> New OLE DB Connection...
New Excel connection	Connection Managers pane -> right-click -> New Connection... -> EXCEL
Open Data Flow tab	Control Flow tab -> double-click a Data Flow Task
Edit Excel Source	Data Flow tab -> double-click Excel Source component
Add derived columns	Data Flow tab -> double-click Derived Column component
Map destination columns	Data Flow tab -> double-click OLE DB Destination -> Mappings page
Edit precedence logic	Control Flow tab -> double-click a precedence arrow
Run the package	Debug menu -> Start Debugging (F5)
Stop debugging	Debug menu -> Stop Debugging (Shift+F5)